PROJECT REPORT

EduOS Memory Manager

Designing a Lightweight OS for Smart Classrooms

By

Mohd Safwan Zain
23WU0102130
BTech 2023-27

Under the esteemed guidance of
Prof. Veeresh Biradar

Table of Contents

Table of Contents 2
1. Introduction 3
2. Problem Statement 3
3. Objectives 3
4. System Design 3
4.1 Paging 3
4.2 Segmentation 4
4.3 Virtual Memory & Demand Paging 4
5. Architecture — Step-by-Step Working 4
6. Page Replacement Algorithms 4
6.1 FIFO — First-In, First-Out 4
6.2 LRU — Least Recently Used 5
6.3 Optimal (OPT / Belady's Algorithm) 5
7. Simulation Results 5
7.1 FIFO Results 5
7.2 LRU Results 6
7.3 Optimal Results 7
8. Comparative Analysis 7
9. Advantages 8
10. Limitations 8
11. Real-World Application — Smart Classrooms 8
12. Conclusion 9
13. GitHub Repository 9
Appendix: Implementation Notes 10
A. Code Corrections Made During Review 10
B. Reference String Used 10

1. Introduction

EduOS is a conceptual lightweight operating system designed exclusively for educational environments, particularly modern Smart Classrooms. With the increasing integration of digital technologies in learning — such as interactive whiteboards, simultaneous student screen sharing, and real-time quiz systems — there is a growing need for an operating system optimised for these distinct workloads.

The memory manager is a crucial component of EduOS, ensuring that educational applications run smoothly, securely, and efficiently on inexpensive, low-power hardware typically found in academic settings.

2. Problem Statement

Standard operating systems (such as Windows or general-purpose Linux distributions) are often heavyweight, featuring background processes and memory requirements that are unnecessary for dedicated learning devices. In a smart classroom, devices must rapidly switch between a few heavy multimedia applications, educational games, and lightweight note-taking apps.

Memory constraints on budget hardware lead to high page fault rates, excessive swapping, and system lag, resulting in disruptions to the learning process. This project designs a dedicated memory manager that minimises overhead while maximising the responsiveness of educational tools.

3. Objectives

1. **Develop a memory manager that maximises the use of limited physical memory on budget classroom devices.** Efficient Resource Allocation:
2. **Minimise the occurrence of page faults when switching between specialised classroom tools (e.g., from PDF reader to video simulation).** Low Latency:
3. **Implement a lightweight architecture using structured paging and virtual memory that does not overwhelm the CPU.** Simplicity and Reliability:
4. **Provide separate memory segments for student and teacher applications to maintain system security and prevent crashes in one application from bringing down the entire system.** Sandbox Isolation:

4. System Design

The EduOS Memory Manager uses a hybrid memory management model, combining the benefits of segmentation and paging, underpinned by a robust virtual memory system.

4.1 Paging

Paging eliminates external fragmentation by dividing physical memory into fixed-size blocks called frames, and logical memory into blocks of the same size called pages. EduOS uses a standard 4

KB page size, which strikes a balance between keeping page tables small and reducing internal fragmentation for small text documents.

4.2 Segmentation

To facilitate security in a school environment, logical memory is segmented by permission level — Student Space, Teacher Space, and Shared Media Space. Each segment is independently paged, so a student application that attempts to read outside its segment boundary triggers a Segment Fault before it can ever reach teacher data.

4.3 Virtual Memory & Demand Paging

EduOS employs demand paging. Since students often have multiple document tabs open simultaneously, the OS loads only the currently required pages of an application into RAM, keeping the remainder on secondary storage (SSD/eMMC) until needed. This allows EduOS to run on devices with as little as 2 GB of RAM without crashing.

5. Architecture — Step-by-Step Working

The following sequence describes how a single memory access request is handled inside EduOS:

5. **A running educational app (e.g., a virtual lab simulator) generates a logical address comprising a Segment Number, a Page Number, and an Offset.**CPU Request:
6. **The MMU verifies the Segment Number against the Segment Table, checking bounds and permissions. A student application attempting to access the Teacher Space segment triggers an immediate access violation.**Segmentation Check:
7. **If the segment check passes, the Page Number is looked up in the Page Table, which maps logical pages to physical frames in RAM.**Page Table Lookup:
8. **A Translation Lookaside Buffer (TLB) caches recent page-to-frame mappings. A TLB hit bypasses the full Page Table walk, reducing latency significantly.**TLB Lookup:
9. **If the target page is not present in physical memory (invalid bit set in the Page Table), a Page Fault interrupt is raised.**Page Fault Handling:
10. **The OS invokes the configured Page Replacement Algorithm to either find a free frame or evict an existing page. The required page is swapped in from disk, the Page Table is updated, and the faulting instruction is restarted transparently.**Page Replacement:

6. Page Replacement Algorithms

To handle memory exhaustion gracefully, EduOS supports three page replacement strategies. The simulation in Section 7 uses the following inputs:

- Reference String (20 accesses): [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
- Number of Physical Frames: 3

6.1 FIFO — First-In, First-Out

The FIFO algorithm evicts the page that has been resident in memory for the longest time. It is straightforward to implement using a queue but suffers from a theoretical weakness called Belady's Anomaly, where increasing the number of frames can paradoxically increase the number of page faults.

In a classroom context, FIFO is a poor fit because it blindly evicts old pages without regard for whether they are still actively being used. A student's open textbook viewer — loaded once and kept open all lesson — could be evicted simply because it was loaded first.

6.2 LRU — Least Recently Used

LRU evicts the page that has not been accessed for the longest period. It exploits temporal locality: pages used recently are statistically likely to be needed again soon. This makes LRU an excellent match for classroom workloads, where students actively toggle between two or three fixed applications throughout a session.

LRU does not suffer from Belady's Anomaly. Its main implementation cost is tracking per-page access timestamps, which EduOS handles using an index counter updated on every page reference.

6.3 Optimal (OPT / Belady's Algorithm)

The Optimal algorithm evicts the page whose next use is farthest in the future (or which will never be used again). It produces the theoretical minimum number of page faults for any given reference string and frame count.

OPT is unimplementable in a real operating system because it requires foreknowledge of the future reference sequence. Its role in EduOS is as a benchmark — it defines the performance ceiling that any practical algorithm should strive toward.

7. Simulation Results

The Python simulation (EduOS_MemoryManager.py) was executed with the 20-element reference string and 3 frames. The detailed step-by-step results for each algorithm are presented below.

7.1 FIFO Results

Page	Frames State	Status / Eviction Reason
7	`[7]`	Fault
0	`[7, 0]`	Fault
1	`[7, 0, 1]`	Fault
2	`[0, 1, 2]`	Fault — evicts 7 (oldest)
0	`[0, 1, 2]`	**Hit**
3	`[1, 2, 3]`	Fault — evicts 0 (oldest)
0	`[2, 3, 0]`	Fault — evicts 1 (oldest)
4	`[3, 0, 4]`	Fault — evicts 2 (oldest)
2	`[0, 4, 2]`	Fault — evicts 3 (oldest)

Page	Frames State	Status / Eviction Reason
3	[4, 2, 3]	Fault — evicts 0 (oldest)
0	[2, 3, 0]	Fault — evicts 4 (oldest)
3	[2, 3, 0]	**Hit**
2	[2, 3, 0]	**Hit**
1	[3, 0, 1]	Fault — evicts 2 (oldest)
2	[0, 1, 2]	Fault — evicts 3 (oldest)
0	[0, 1, 2]	**Hit**
1	[0, 1, 2]	**Hit**
7	[1, 2, 7]	Fault — evicts 0 (oldest)
0	[2, 7, 0]	Fault — evicts 1 (oldest)
1	[7, 0, 1]	Fault — evicts 2 (oldest)

Total Page Faults (FIFO): 15

7.2 LRU Results

Page	Frames State	Status / Eviction Reason
7	[7]	Fault
0	[7, 0]	Fault
1	[7, 0, 1]	Fault
2	[0, 1, 2]	Fault — evicts 7 (LRU)
0	[0, 1, 2]	**Hit**
3	[0, 2, 3]	Fault — evicts 1 (LRU)
0	[0, 2, 3]	**Hit**
4	[0, 3, 4]	Fault — evicts 2 (LRU)
2	[0, 4, 2]	Fault — evicts 3 (LRU)
3	[4, 2, 3]	Fault — evicts 0 (LRU)
0	[2, 3, 0]	Fault — evicts 4 (LRU)
3	[2, 3, 0]	**Hit**
2	[2, 3, 0]	**Hit**
1	[2, 3, 1]	Fault — evicts 0 (LRU)
2	[2, 3, 1]	**Hit**
0	[2, 1, 0]	Fault — evicts 3 (LRU)
1	[2, 1, 0]	**Hit**
7	[1, 0, 7]	Fault — evicts 2 (LRU)
0	[1, 0, 7]	**Hit**
1	[1, 0, 7]	**Hit**

Total Page Faults (LRU): 12

7.3 Optimal Results

Page	Frames State	Status / Eviction Reason
7	[7]	Fault
0	[7, 0]	Fault
1	[7, 0, 1]	Fault
2	[0, 1, 2]	Fault — evicts 7 (never used again)
0	[0, 1, 2]	**Hit**
3	[0, 2, 3]	Fault — evicts 1 (furthest future use)
0	[0, 2, 3]	**Hit**
4	[2, 3, 4]	Fault — evicts 0 (furthest future use)
2	[2, 3, 4]	**Hit**
3	[2, 3, 4]	**Hit**
0	[2, 3, 0]	Fault — evicts 4 (never used again)
3	[2, 3, 0]	**Hit**
2	[2, 3, 0]	**Hit**
1	[2, 0, 1]	Fault — evicts 3 (furthest future use)
2	[2, 0, 1]	**Hit**
0	[2, 0, 1]	**Hit**
1	[2, 0, 1]	**Hit**
7	[0, 1, 7]	Fault — evicts 2 (never used again)
0	[0, 1, 7]	**Hit**
1	[0, 1, 7]	**Hit**

Total Page Faults (Optimal): 9

8. Comparative Analysis

Algorithm	Page Faults	Extra vs OPT	Notes
FIFO	15	+6	Simple – no extra hardware needed
LRU	12	+3	Best practical – exploits temporal locality
Optimal	9	0	Theoretical lower bound – unimplementable

The results confirm the theoretical expectations of all three algorithms. FIFO's 15 faults (67% above optimal) demonstrate its inability to exploit access patterns. LRU's 12 faults (33% above optimal) show a meaningful improvement by leveraging temporal locality. The Optimal algorithm's 9 faults establish the performance ceiling.

For EduOS, LRU is the recommended production algorithm. The 3-fault gap from Optimal is an acceptable cost for a practically implementable strategy, and LRU's performance advantage over FIFO directly translates to fewer disruptive page-load delays during classroom sessions.

9. Advantages

- Tailored Performance: Removes the bloat of traditional OS memory managers, leading to faster application load times on inexpensive student hardware.
- No External Fragmentation: Fixed-size paging completely eliminates external fragmentation, preserving all available physical memory.
- Logical Security: Application segmentation natively separates instructor tools from student processes, mitigating unauthorised memory access at the hardware level.
- Hardware-Friendly: Runs sufficiently on devices with 2–4 GB of RAM due to aggressive demand-paging using the LRU algorithm.

10. Limitations

- Internal Fragmentation: Paging inevitably wastes some memory at the end of the last page allocated to a process.
- Table Overhead: Maintaining both segment tables and page tables consumes a portion of the very RAM they manage.
- TLB Miss Penalty: Switching between processes with large, dissimilar memory footprints incurs higher TLB miss rates, adding context-switch latency.
- LRU Implementation Cost: Tracking per-page timestamps requires additional bookkeeping that must be kept fast to avoid negating LRU's performance gains.

11. Real-World Application — Smart Classrooms

In a modern smart classroom, every student desk has an interactive tablet, and the teacher operates a central smartboard. EduOS enables the following scenarios:

- A student keeps a heavy geometric simulation active while simultaneously taking notes. When memory pressure rises, EduOS gracefully pages out background app data rather than terminating the foreground process.
- Teachers run grading and classroom management applications in isolated Teacher Space segments that cannot be read or written by any student application, regardless of how the student app behaves.

- The system maintains a distraction-limiting environment — no unnecessary background services consuming RAM — so students never experience disruptive lag or thermal throttling during lessons.

12. Conclusion

The EduOS Memory Manager bridges the gap between the hardware limitations of budget educational devices and the demanding requirements of modern classroom software. By combining segmentation-based isolation with demand paging and the LRU replacement policy, EduOS achieves an optimal balance of security, responsiveness, and memory efficiency.

The simulation results validate this design choice quantitatively: LRU reduces page faults by 20% compared to FIFO on a representative classroom access sequence, coming within 33% of the theoretically perfect Optimal algorithm. This translates directly into smoother, uninterrupted learning experiences without requiring expensive hardware investments.

13. GitHub Repository

The complete source code for this project — including the Python simulation, documentation, and output logs — is publicly available on GitHub. You can browse the repository, clone it locally, or submit issues using the link below.

Repository: https://github.com/safwanwaris/EduOS-Memory-Manager

Scan to visit the repository:

`github.com/safwanwaris/EduOS-Memory-Manager`

Appendix: Implementation Notes

A. Code Corrections Made During Review

The following issues were identified in the original code and corrected in the submitted version:

11. **The original used recent_usage.get(x, -1) as the default for pages not yet in the dictionary. This meant unloaded pages could be treated as the most recently used, causing incorrect evictions. Fixed by ensuring recent_usage is only queried for pages already in frames (all of which are guaranteed to be in the dictionary at eviction time).**LRU eviction bug:
12. **The original report Section 6 described a 7-element reference string (7, 0, 1, 2, 0, 3, 0) with fault counts of 6/5/4, while the simulation used a 20-element string producing 15/12/9. The report has been updated to consistently use the 20-element simulation reference string throughout.**Report/code mismatch:
13. **Added a validate_inputs() function that raises clear errors for empty reference strings or non-positive frame counts, preventing silent incorrect results.**Input validation:
14. **Replaced the plain-text summary with a structured comparison table showing faults and deviation from Optimal, making the analysis immediately actionable.**Summary output:
15. **Added full PEP 484 type annotations and detailed docstrings to all functions for maintainability and clarity.**Type hints and docstrings:

B. Reference String Used

The simulation uses the following 20-element reference string representing a student alternating between classroom applications across a lesson:

```
[7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
```

Number of physical frames: 3 (representative of budget classroom hardware with limited RAM).